

ARRAY PATTERN HANDBOOK

Pattern 1 : Prefix Sum

Recognition Signals

Question contains:

- Subarray Sum
- Range Sum
- Sum Equals K
- Continuous Sum
- Running Total

Think:

$$\text{prefix}[i] = \text{prefix}[i-1] + \text{nums}[i]$$

Core Observation

Instead of calculating every subarray sum:

$$\text{sum}(i,j)$$

Use:

$$\text{prefix}[j] - \text{prefix}[i-1]$$

Invariant

$$\text{prefix}[i]$$

=

sum of elements from 0 to i

Never changes.

Generic Template

```
prefix = 0

for(num : nums){

    prefix += num

}
```

Variations

Type 1

Range Sum Query

Example:

- Range Sum Query
 - Pivot Index
-

Type 2

Prefix Sum + HashMap

Example:

- Subarray Sum Equals K
 - Continuous Subarray Sum
-

Type 3

Prefix Difference

Example:

- Find Longest Equal 0s and 1s
-

Common Problems

Easy

- Running Sum

- Pivot Index

Medium

- Subarray Sum Equals K
 - Continuous Subarray Sum
 - Contiguous Array
-

Quick Trigger

Need many subarray sums?

→ Prefix Sum

Pattern 2 : Two Pointers

Recognition Signals

Question contains:

- Sorted Array
- Pair Sum
- Triplet Sum
- Opposite Ends
- Remove Duplicates
- Palindrome

Think:

left
right

Core Observation

Two indices move intelligently.

Avoid:

$O(n^2)$

Convert into:

$O(n)$

Invariant

Everything outside:

[left,right]

already processed.

Generic Template

```
left=0;
```

```
right=n-1;
```

```
while(left<right){
```

```
}
```

Variations

Type 1

Opposite Direction

Examples

- Two Sum II
 - Container With Most Water
 - Trapping Rain Water
-

Type 2

Same Direction

Examples

- Remove Duplicates
 - Move Zeroes
-

Type 3

Sorting + Two Pointers

Examples

- 3Sum
 - 4Sum
 - Triangle Number
-

Common Problems

Easy

- Move Zeroes
- Remove Duplicates

Medium

- 3Sum
- Container With Most Water

Hard

- Trapping Rain Water
-

Quick Trigger

Sorted?

Pair?

Triplet?

Both ends?

→ Two Pointers

Pattern 3 : Sliding Window

Recognition Signals

Question contains:

- Longest
- Shortest
- Maximum
- Minimum

AND

contiguous subarray

Core Observation

Expand window.

Shrink when invalid.

Invariant

Current window:

[left,right]

always represents candidate answer.

Generic Template

```
for(right){  
  
    add(right)  
  
    while(invalid){  
  
        remove(left)  
        left++  
    }  
  
    update answer  
}
```

Variations

Type 1

Fixed Window

Examples

- Maximum Average Subarray
-

Type 2

Variable Window

Examples

- Longest Repeating Character Replacement
 - Minimum Size Subarray Sum
-

Type 3

Sliding Window + HashMap

Examples

- Fruit Into Baskets
 - Subarrays with K Distinct
-

Common Problems

Easy

- Maximum Average Subarray

Medium

- Fruit Into Baskets
- Longest Substring Without Repeating

Hard

- Minimum Window Substring
-

Quick Trigger

Longest?

Shortest?
Contiguous?

→ Sliding Window

Pattern 4 : Hashing + Arrays

Recognition Signals

Question contains:

- Frequency
 - Duplicate
 - Count Occurrences
 - Lookup Fast
 - Need $O(1)$
-

Core Observation

Trade memory for speed.

Invariant

HashMap always stores information already seen.

Generic Template

`Map<Integer,Integer> map`

or

`Set<Integer> set`

Variations

Type 1

Frequency Counting

Examples

- Top K Frequent
 - Majority Element
-

Type 2

Visited Elements

Examples

- Contains Duplicate
 - Longest Consecutive Sequence
-

Type 3

HashMap + Prefix

Examples

- Subarray Sum Equals K
-

Quick Trigger

Need fast lookup?

→ HashMap

Pattern 5 : Kadane Pattern

Recognition Signals

Question contains:

- Maximum Sum Subarray
- Best Continuous Segment
- Largest Gain

Core Observation

Negative prefix hurts future answer.

Discard it.

Invariant

currentSum

=

Best subarray ending at current index.

Generic Template

current+=num

current=Math.max(current,num)

best=Math.max(best,current)

Variations

Type 1

Maximum Sum

- Maximum Subarray

Type 2

Minimum Sum

- Minimum Subarray

Type 3

Circular Kadane

- Maximum Circular Subarray

Quick Trigger

Maximum contiguous sum?

→ Kadane

Pattern 6 : Partitioning (Dutch National Flag)

Recognition Signals

Question contains:

- Sort 0 1 2
 - Three Regions
 - Partition Around Pivot
-

Core Observation

Maintain regions.

Invariant

0 → low-1

correct

low → mid-1

correct

mid → high

unknown

high+1 → end

correct

Generic Template

low
mid
high

Problems

- Sort Colors
 - Wiggle Sort
 - Quick Sort Partition
-

Quick Trigger

Multiple categories?

Partition?

→ Dutch National Flag

ARRAY MASTER IDENTIFICATION CHEATSHEET

Subarray Sum?

→ Prefix Sum

Longest/Shortest Contiguous?

→ Sliding Window

Pair / Triplet?

→ Two Pointers

Need Frequency?

→ HashMap

Maximum Continuous Sum?

→ Kadane

Sort 0 1 2?

→ Partitioning

Duplicates?

→ Hashing

Range Sum?

→ Prefix Sum

ARRAY PATTERN CHEAT SHEET

Visual Patterns, Types & Templates

1. PREFIX SUM PATTERN

VISUAL MODEL

Index :	0	1	2	3	4	5
Array :	1	2	3	4	5	6
Prefix :	1	3	6	10	15	21

Subarray Sum (L..R)
= Prefix[R] - Prefix[L-1]



Sum = Prefix[3] - Prefix[0] = 10 - 1 = 9

RECOGNITION

- Subarray Sum
- Range Sum
- Sum = K
- Running Total

TYPES

1. Simple Prefix

prefix[i] = sum[0..i]

Ex: Running Sum
Pivot Index

2. Prefix + HashMap

Need: prefix - k

Ex: Subarray Sum = K
Continuous Subarray Sum

3. Prefix Difference

Map diff of prefix

Ex: Contiguous Array
(0s and 1s)

TEMPLATE

```
int prefix = 0;
for(int i = 0; i < n; i++){
    prefix += nums[i];
    // use prefix
}
```

```
// With HashMap
Map<Integer, Integer> map = new HashMap<>();
map.put(0, 1);
int prefix = 0;
```

2. TWO POINTER PATTERN

VISUAL MODEL



RECOGNITION

- Sorted Array
- Pair / Triplet Sum
- Opposite Ends
- Remove Duplicates
- Palindrome

TYPES

1. Opposite Direction

left → right ←

Move towards each other

Ex: Two Sum II
Container With Most Water
Trapping Rain Water

2. Same Direction

slow → fast

Remove/Skip Duplicates, Move Zeroes

Ex: Remove Duplicates
Move Zeroes

3. Sorting + Two Pointers

Sort →

Fix one, use two pointers

Ex: 3Sum
4Sum
Triangle Count

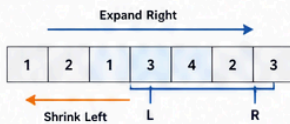
TEMPLATE

```
// Opposite Direction
int left = 0, right = n - 1;
while(left < right){
    // logic
}
```

```
// Same Direction
int i = 0, j = 1;
while(j < n){
    // logic
}
```

3. SLIDING WINDOW PATTERN

VISUAL MODEL



RECOGNITION

- Longest / Shortest
- Maximum / Minimum
- Contiguous Subarray
- K Distinct / Sum >= K

TYPES

1. Fixed Window

Window size = K

Ex: Max Average Subarray of Size K

2. Variable Window

Expand & shrink based on condition

Ex: Longest Repeating Character Replacement
Min Size Subarray Sum

3. Window + HashMap

Maintain frequency in window

Ex: Fruit Into Baskets
Subarrays with K Distinct

TEMPLATE

```
int left = 0;
for(int right = 0; right < n; right++){
    add(nums[right]);
    while(invalid condition){
        remove(nums[left]);
        left++;
    }
    // update answer
}
```

add() → include nums[right]
remove() → remove nums[left]
invalid → window not valid
update → answer update

4. HASHING PATTERN (Arrays)

VISUAL MODEL

Array:	4	1	2	1	3	2	4
HashMap (freq)	1	2	3	4			
	2	2	1	2			

RECOGNITION

- Frequency Count
- Duplicate / Unique
- Need Fast Lookup
- Count Occurrences

TYPES

1. Frequency Counting

Count frequency of elements

Ex: Top K Frequent
Majority Element
Sort by Frequency

2. Seen / Existence

Check if element already seen

Ex: Contains Duplicate
Longest Consecutive Sequence

3. HashMap + Prefix

Store prefix in map for fast lookup

Ex: Subarray Sum = K
Longest Subarray with Sum K

TEMPLATE

```
// Frequency
Map<Integer, Integer> map = new HashMap<>();
for(int num : nums){
    map.put(num, map.getOrDefault(num, 0) + 1);
}
```

```
// Seen
Set<Integer> set = new HashSet<>();
for(int num : nums){
    if(set.contains(num)) { /* found */ }
    set.add(num);
}
```

5. KADANE'S PATTERN (Max Subarray)

VISUAL MODEL

Array:	-2	1	-3	4	-1	2	1	-5	4
Best Subarray →				4	-1	2	1		
Max Sum =				6					

RECOGNITION

- Maximum Subarray Sum
- Best Continuous Sum
- Largest Gain

TYPES

1. Maximum Subarray

Normal Kadane

Ex: Maximum Subarray

2. Minimum Subarray

Apply Kadane on negative numbers

Ex: Minimum Subarray

3. Circular Subarray

Total Sum - Min Subarray

Ex: Maximum Circular Subarray

TEMPLATE

```
int best = nums[0];
int curr = nums[0];
for(int i = 1; i < n; i++){
    curr = Math.max(nums[i], curr + nums[i]);
    best = Math.max(best, curr);
}
return best;
```

Idea:
If current sum < 0,
drop it and start new
from current element.

6. DUTCH NATIONAL FLAG (Partitioning)

VISUAL MODEL (Sort 0,1,2)



RECOGNITION

- Sort 0,1,2
- Partition into three parts
- Three way partition

TYPES / USES

- Sort Colors (0,1,2)
- Partition Around Pivot
- Hashing
- Used in QuickSort (Partition step)

TEMPLATE

```
int low = 0, mid = 0, high = n - 1;
while(mid < high){
    if(nums[mid] == 0){
        swap(nums[low++], nums[mid++]);
    } else if(nums[mid] == 2){
        swap(nums[mid], nums[high--]);
    } else {
        mid++;
    }
}
```

Regions Invariant

0 .. low-1 → all 0s
low ... mid-1 → all 1s
mid ... high → unknown
high+1 ... n-1 → all 2s

QUICK PATTERN IDENTIFICATION



Subarray Sum / Range Sum
→ Prefix Sum



Pair / Triplet /
Opposite Ends
→ Two Pointers



Longest / Shortest
Contiguous
→ Sliding Window



Need Frequency /
Lookup
→ Hashing



Max Continuous Sum
→ Kadane



Sort 0,1,2 /
Partition
→ Dutch Flag